DATA STRUCTURES & ALGORITHMS · C++

Hashing in C++ From Direct Addressing to Placement-Ready

Everything for hashing: how a hash table works inside, chaining and open addressing by hand, unordered_map versus map, the traps in operator[], frequency counting, complement lookups, prefix sums with a hash map, sliding windows, LRU and O(1) designs, rolling hashes, the 12 patterns, and a curated problem list.

59	12	21	38
SECTIONS	PATTERNS	CODE BLOCKS	PROBLEMS

Prepared for **Hari** · on-campus placement preparation
Generated 14 October 2026

Contents

PART 0 — Hashing, from absolute zero

0.1 Start with direct addressing

0.2 The hash function — squeeze any key into a small array

0.3 Collisions are unavoidable

0.4 Load factor, and why "O(1)" is an average

0.5 The vocabulary

0.6 How to recognise the topic in a problem statement

PART 1 — Building a hash table by hand

1.1 Separate chaining

1.2 Open addressing with linear probing

1.3 Why deletion needs tombstones

1.4 Probing strategies, and clustering

1.5 What makes a good hash function

PART 2 — The STL

2.1 The four containers

2.2 The operations, and how to use them correctly

2.3 The operator[] trap

2.4 Erasing while iterating

2.5 Keys that are not built-in types

2.6 Anti-hash tests, and reserve

2.7 Often, an array is the best hash map

PART 3 — The hashing mental model

3.1 "Remember what you have seen, so you never search for it again"

3.2 Three questions that design the solution

3.3 Hash table, sorting, or two pointers?

PART 4 — Counting and frequency

4.1 The counting map

4.2 Counting pairs in one pass

4.3 Signatures — grouping things that are "the same"

PART 5 — Seen-before and complement lookups

5.1 Two Sum — the most important problem in the topic

5.2 Duplicates, and duplicates within a distance

5.3 Detecting a cycle with a set

5.4 Longest Consecutive Sequence (LC 128) — the O(n) trick

5.5 Meet in the middle — 4Sum II (LC 454)

PART 6 — Prefix sums plus a hash map

6.1 The idea

6.2 The variants, and what changes

6.3 Negative remainders

6.4 Lifting to two dimensions (LC 1074)

PART 7 — Sliding windows with a hash map

7.1 Longest substring without repeating characters (LC 3)

7.2 Fixed-size window: Find All Anagrams (LC 438)

7.3 Minimum window containing all of t (LC 76)

7.4 Exactly k = at most k − at most (k − 1)

PART 8 — Mappings and bijections

8.1 Isomorphic strings and word pattern

8.2 The first-occurrence signature

PART 9 — Design problems: O(1) with a hash map plus something else

9.1 LRU Cache (LC 146) — hash map + doubly linked list

9.2 Insert, Delete, GetRandom in O(1) (LC 380)

9.3 LFU Cache (LC 460)

9.4 Design HashMap (LC 706)

PART 10 — Hashing strings: rolling hashes

10.1 Why a rolling hash

10.2 The polynomial rolling hash

10.3 Rabin-Karp: find every occurrence of a pattern

10.4 Collisions, and how to make them unlikely

10.5 When the hash can be perfect

10.6 Binary search on the answer + rolling hash

PART 11 — The array as its own hash table

11.1 Cyclic placement — First Missing Positive (LC 41)

11.2 Sign marking

PART 12 — The complete pattern taxonomy

How to tell 3 and 7 apart

How to tell 7 and 8 apart

PART 13 — How to approach ANY hashing problem

Step 1 — Write the brute force and find the inner search

Step 2 — Choose the key

Step 3 — Choose the value: presence, count, first index or last index

Step 4 — Decide the seed and the order

Step 5 — Guard the lookups

Step 6 — Dry-run the small cases

PART 14 — The bug catalogue

PART 15 — Complexity cheat sheet

PART 16 — The problem list

PART 17 — A three-week plan

Quick reference card

PART 0 — Hashing, from absolute zero

Every data structure so far answered "*where is the element at position i?*" A hash table answers a different question: **"have I seen this key, and what did I store with it?"** — in O(1) time on average, no matter how many keys there are.

That one ability changes the complexity of a huge number of problems. The nested loop that asks "*is there an earlier element equal to X?*" becomes a single lookup. $O(n^2)$ becomes O(n). That is why hashing is the most frequently used tool in coding interviews, and why the first problem on LeetCode — Two Sum — is a hashing problem.

> **The single sentence to remember:** hashing trades memory for time. You remember what you have seen, so you never have to search for it again.

0.1 Start with direct addressing

Suppose the keys are small non-negative integers — exam marks from 0 to 100. You do not need anything clever: make an array of 101 slots and use the key **as the index**.

```
int count[101] = {0};
for (int mark : marks) count[mark]++;        // O(1) per mark
cout << count[87];                           // how many scored 87? O(1)
```

This is **direct addressing**, and it is perfect: every operation is O(1), worst case, no collisions. You have used it already — `int cnt[26]` for letters is direct addressing.

It breaks down in two situations:

- **The key range is huge.** Phone numbers, values up to 10^9, 64-bit IDs — an array that big does not fit in memory.
- **The keys are not integers.** Strings, pairs, structs.

0.2 The hash function — squeeze any key into a small array

A **hash function** turns any key into an index in a table of B slots, called **buckets**:

```
key ──► hash function ──► index in 0 … B−1

"apple"    ──►  h("apple")  % 8  =  3
1000000007 ──►  1000000007  % 8  =  7
42         ──►  42          % 8  =  2
```

Now a table of B slots can hold keys from an enormous range. The cost: two different keys can land in the **same** bucket.

0.3 Collisions are unavoidable

There are more possible keys than buckets, so by the **pigeonhole principle** some keys must share a bucket. That is a **collision**, and every hash table is defined by how it handles them. There are two

families, both built by hand in Part 1:

- **Separate chaining** — each bucket holds a small list of every key that hashed there.
- **Open addressing** — every key lives in the array itself; on a collision, probe for another free slot.

```
separate chaining, B = 8, keys 1, 9, 17, 4

bucket 0 │ ·
bucket 1 │ ──► 1 ──► 9 ──► 17          (all three hash to 1)
bucket 2 │ ·
bucket 3 │ ·
bucket 4 │ ──► 4
bucket 5 │ ·
bucket 6 │ ·
bucket 7 │ ·
```

0.4 Load factor, and why "O(1)" is an average

The **load factor** is α = n / B — keys per bucket on average.

- With a decent hash function, keys spread evenly and each bucket holds about α keys, so a lookup costs **O(1 + α)**.
- The table **rehashes** — allocates more buckets (typically double) and re-inserts everything — whenever α passes a threshold. That keeps α ≤ 1, so operations stay **O(1) on average**.
- Rehashing one time costs O(n), but it happens after n insertions, so its cost spreads to **amortised O(1)** per insert — the same argument as `vector::push_back`.

The **worst case is O(n)**: if every key collides into one bucket, a lookup walks a list of n keys. That happens with a bad hash function — or with an adversary who knows yours (Part 2.6).

	AVERAGE	WORST
insert / find / erase	**O(1)**	O(n)
rehash (all keys)	O(n), amortised O(1) per insert	

0.5 The vocabulary

TERM	MEANING
Hash function	Maps a key to an integer; `% B` then picks the bucket
Bucket	One slot of the table
Collision	Two different keys mapping to the same bucket
Separate chaining	Each bucket holds a list of colliding keys
Open addressing	Keys live in the array; collisions probe for another slot
Linear / quadratic probing, double hashing	Three ways to choose the next slot to try
Tombstone	A "deleted" marker in open addressing, so probe chains are not broken
Load factor α	n / B, the average number of keys per bucket
Rehashing	Growing the table and re-inserting every key
Hash set / hash map	Stores keys only / stores key → value pairs
Signature	A key built so that "equivalent" items share it — e.g. the sorted letters of an anagram
Rolling hash	A string hash that can slide one character in O(1) (Part 10)

0.6 How to recognise the topic in a problem statement

THE PROBLEM SAYS	REACH FOR
"have I seen this before", "duplicate", "unique", "first repeating"	`unordered_set`
"how many times", "frequency", "most common", "anagram"	counting map, or `int cnt[26]`
"two numbers that add up to", "pair with difference k", "complement"	value → index map
"subarray with sum k", "equal number of 0s and 1s", "divisible by k"	**prefix sum + hash map**
"longest substring without …", "window with at most k distinct"	sliding window + map
"group the …", "same pattern", "isomorphic", "can be rearranged into"	signature as the key
"design … in O(1)" — LRU cache, insert / delete / getRandom	hash map + list or vector
"find a repeated substring", "pattern matching"	rolling hash
"first missing positive" with O(1) extra space	the array as its own hash table
an $O(n^2)$ brute force that checks "does an earlier element satisfy X?"	almost always hash the earlier elements

That last row is the real signal. **Whenever your brute force has an inner loop that searches the elements you have already passed, a hash table can usually replace the inner loop with one lookup.**

PART 1 — Building a hash table by hand

Written rounds ask about collision handling constantly — "*explain chaining and open addressing*", "*why do we need tombstones?*", "*what is the load factor?*" Building both kinds once makes all of those answers obvious.

1.1 Separate chaining

```
class ChainedHashMap {
    vector<list<pair<int, int>>> buckets = vector<list<pair<int, int>>>(8);
    int count = 0;
    int idx(int key) const {
        int B = buckets.size();
        return ((key % B) + B) % B;           // C++ % can be negative: fix it
    }
    void rehash() {
        vector<list<pair<int, int>>> old(buckets.size() * 2);
        swap(old, buckets);                   // buckets is now the larger table
        for (auto& chain : old)
            for (auto& kv : chain) buckets[idx(kv.first)].push_back(kv);
    }
public:
    void put(int key, int value) {
        for (auto& kv : buckets[idx(key)])
            if (kv.first == key) { kv.second = value; return; }   // update in place
        if (count + 1 > (int)buckets.size()) rehash();           // keep α <= 1
        buckets[idx(key)].push_back({key, value});
        count++;
    }
    int get(int key) {
        for (auto& kv : buckets[idx(key)]) if (kv.first == key) return kv.second;
        return -1;
    }
};
```

Three details worth stating in an interview:

- `((key % B) + B) % B` — in C++, `-7 % 8` is `-7`, not `1`. A negative index is undefined behaviour.
- **Rehashing moves keys.** A key's bucket depends on B, so after doubling, every key must be re-inserted with the new `idx`. You cannot just copy the bucket array.
- **Prime bucket counts** reduce clustering when keys share a common factor (all multiples of 8 land in bucket 0 of an 8-bucket table). `std::unordered_map` uses primes; power-of-two tables instead mix the hash bits first.

This is exactly how `std::unordered_map` is organised: an array of buckets, each a chain of nodes.

1.2 Open addressing with linear probing

All keys live in one flat array. To insert, go to the home slot; if it is taken, try the next one, and the next, wrapping around.

```
64 slots. Insert 5, then 69, then 133 — all have home slot 5.

slot:    4      5      6      7      8
      [     ][  5  ][  69 ][ 133 ][     ]
               home    +1     +2

contains(133): start at 5, skip 5 and 69, find 133 at slot 7.
contains(197): start at 5, skip 5, 69, 133, reach EMPTY slot 8 -> not present.
```

The search stops at the first **empty** slot, because if the key existed it would have been placed there or earlier. That stopping rule is what makes deletion tricky.

1.3 Why deletion needs tombstones

Erase 69 by setting slot 6 back to EMPTY, then look for 133:

```
slot:    4      5      6      7      8
      [     ][  5  ][EMPTY][ 133 ][     ]

contains(133): start at 5, skip 5, reach EMPTY slot 6 -> "not present"   WRONG
```

The empty slot **broke the probe chain**. The fix: mark erased slots **DELETED** (a *tombstone*). Searches walk past tombstones; inserts may reuse them.

```cpp
enum State { EMPTY, FULL, DELETED };

bool contains(int x) {
    for (int i = home(x), steps = 0; steps < CAP; i = (i + 1) % CAP, steps++) {
        if (state[i] == EMPTY) return false;              // true end of the chain
        if (state[i] == FULL && keys[i] == x) return true;
    }                                                     // DELETED: keep walking
    return false;
}
bool erase(int x) {
    for (int i = home(x), steps = 0; steps < CAP; i = (i + 1) % CAP, steps++) {
        if (state[i] == EMPTY) return false;
        if (state[i] == FULL && keys[i] == x) { state[i] = DELETED; return true; }
    }
    return false;
}
```

Tombstones accumulate and slow searches down, so real implementations rebuild the table when tombstones plus keys pass a threshold.

1.4 Probing strategies, and clustering

STRATEGY	NEXT SLOT TO TRY	PROBLEM
Linear probing	h, h+1, h+2, …	**primary clustering**: long runs of full slots grow, and every key landing in a run extends it
Quadratic probing	h, $h+1^2$, $h+2^2$, …	**secondary clustering**: keys with the same home follow the same path
Double hashing	h, h + h_2, h + 2·h_2, …	best spread; needs a second hash function, and h_2 must never be 0

Open addressing must keep the load factor **well below 1** (typically 0.5–0.7), because a full table has nowhere to probe. Chaining tolerates α > 1 gracefully.

	SEPARATE CHAINING	OPEN ADDRESSING
Memory	a node per key (pointers)	one flat array, cache-friendly
Load factor	can exceed 1	must stay < 1
Deletion	easy — unlink the node	needs tombstones
Worst case	long chain	long probe run
Used by	`std::unordered_map`, Java `HashMap`	Python `dict`, Google's `absl::flat_hash_map`, Rust `HashMap`

1.5 What makes a good hash function

- **Deterministic** — the same key always gives the same hash.
- **Uniform** — keys spread evenly over the buckets.
- **Fast** — it runs on every operation.
- **Uses the whole key** — a string hash that only looked at the first character would put every word starting with `a` in one bucket.

For strings the standard choice is the **polynomial hash**:

```
h(s) = s[0]·B^(n−1) + s[1]·B^(n−2) + … + s[n−1]      (mod M)

computed left to right:   h = h * B + s[i]   (mod M)
```

with B a small prime such as 31 or 131 and M a large prime such as $10^9 + 7$. Order matters — `"ab"` and `"ba"` hash differently, unlike a plain sum of characters. Part 10 makes this hash *roll*.

PART 2 — The STL

2.1 The four containers

CONTAINER	STRUCTURE	FIND / INSERT / ERASE	ORDERED?	HEADER
`unordered_set<K>`	hash table	**O(1) avg**, O(n) worst	no	`<unordered_set>`
`unordered_map<K, V>`	hash table	**O(1) avg**, O(n) worst	no	`<unordered_map>`
`set<K>`	red-black tree	O(log n) always	**yes**	`<set>`
`map<K, V>`	red-black tree	O(log n) always	**yes**	`<map>`

Plus `unordered_multiset` / `unordered_multimap` / `multiset` / `multimap`, which allow repeated keys.

When to choose the tree versions: you need keys **in order** (iterate sorted, smallest / largest), you need `lower_bound` / `upper_bound` ("the first key ≥ x"), or you need a guaranteed O(log n) worst case. Otherwise the hash versions are faster.

2.2 The operations, and how to use them correctly

```
unordered_map<string, int> m;

m["apple"] = 3;                 // insert or overwrite
m["apple"]++;                   // read-modify-write; creates 0 first if absent

m.count("pear");                // 0 or 1 — "is it there?"
m.contains("pear");             // C++20 — same thing, clearer

auto it = m.find("pear");       // iterator, or m.end()
if (it != m.end()) use(it->second);   // ONE lookup for check-and-read

m.insert({"kiwi", 5});          // does NOT overwrite an existing key
auto [pos, inserted] = m.insert({"kiwi", 9});   // inserted == false, value still 5

m.erase("apple");               // by key — returns how many were removed (0 or 1)
m.erase(it);                    // by iterator
m.size(); m.empty(); m.clear();

for (auto& [key, value] : m) { ... }   // structured bindings, ANY order
```

2.3 The `operator[]` trap

`m[key]` **inserts the key with a default value (0, "", empty vector) if it is not there.** That is convenient for counting — `cnt[x]++` just works — and a bug everywhere else.

```
if (m[key] > 0) { ... }    // BUG: inserts key with 0 when absent
                           //  - the map grows on every miss
                           //  - m.size() now counts keys you never added
                           //  - it modifies the map while you iterate it

if (m.count(key) && m[key] > 0) { ... }        // OK, two lookups
auto it = m.find(key);                          // best, one lookup
if (it != m.end() && it->second > 0) { ... }
```

`operator[]` also does not compile on a `const` map. Rule of thumb: **use `[]` to write, use `find` or `count` to read.**

2.4 Erasing while iterating

```
for (auto it = m.begin(); it != m.end(); ) {
    if (it->second == 0) it = m.erase(it);    // erase returns the next iterator
    else ++it;
}
```

`m.erase(it); ++it;` uses an iterator that was just destroyed — undefined behaviour. And never **insert** into an unordered container while iterating it: a rehash invalidates every iterator.

2.5 Keys that are not built-in types

`unordered_map` needs `std::hash<K>`. It exists for integers, `string`, pointers and `char` — but **not for `pair`, `vector` or your own structs.** Four ways round it, in order of preference in an interview:

```
// 1. Encode the key as one integer (fastest; works when ranges are known)
long long key = (long long)r * 100000 + c;
unordered_set<long long> seen;

// 2. Encode the key as a string
string key = to_string(a) + "," + to_string(b);

// 3. Use the ordered map: map / set compare pairs and vectors out of the box
map<pair<int, int>, int> cnt;                    // O(log n), no hash needed

// 4. Write a hash functor
struct PairHash {
    size_t operator()(const pair<int, int>& p) const {
        return hash<long long>()(((long long)p.first << 32) ^ (unsigned)p.second);
    }
};
unordered_map<pair<int, int>, int, PairHash> m;
```

2.6 Anti-hash tests, and `reserve`

`std::hash<int>` is the identity on GCC, and the bucket counts are a known sequence of primes. On Codeforces, people deliberately submit inputs where every key is a multiple of the bucket count, turning every operation into O(n) — your accepted solution then TLEs during hacking. LeetCode and on-campus judges do not do this, but know the defence:

```
struct SplitMix {                                // a well-mixed hash with a random seed
    static uint64_t mix(uint64_t x) {
        x += 0x9e3779b97f4a7c15ULL;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ULL;
        x = (x ^ (x >> 27)) * 0x94d049bb133111ebULL;
        return x ^ (x >> 31);
    }
    size_t operator()(uint64_t x) const {
        static const uint64_t seed = chrono::steady_clock::now().time_since_epoch().count();
        return mix(x + seed);
    }
};
unordered_map<long long, int, SplitMix> safe;
```

Separately, if you know roughly how many keys you will insert, `m.reserve(n)` allocates the buckets once and avoids every intermediate rehash — a free 2× speed-up on large inputs.

2.7 Often, an array is the best hash map

When the keys are small integers or characters, a plain array is faster, simpler and has no worst case:

KEYS	USE
lowercase letters	`int cnt[26]`, index `c - 'a'`
any ASCII character	`int cnt[128]` or `int cnt[256]`
values in 0 … n	`vector<int> cnt(n + 1)`
remainders mod k	`vector<int> cnt(k)`
booleans for a 9 × 9 board	`bool seen[9][10]`

`int cnt[26]` also compares in one call — `equal(a, a + 26, b)` — and can be turned into a string key for grouping. Reach for it first whenever the alphabet is fixed.

PART 3 — The hashing mental model

3.1 "Remember what you have seen, so you never search for it again"

Almost every hashing solution is a **single left-to-right pass** with a hash table holding information about the elements **already passed**. At each new element you ask one question of the table, then you add the element to it.

```
for (int i = 0; i < n; i++) {
    <ask the table a question about a[i]>     // 1. LOOK UP
    <record a[i] in the table>                // 2. THEN INSERT
}
```

The order of those two lines matters. Looking up **before** inserting means the table contains only *earlier* elements, so an element can never be paired with itself — which is exactly what Two Sum with `[3,3]` and target 6 needs, and what "pairs i < j" means in general.

3.2 Three questions that design the solution

1. **What is the key?** The value itself? A prefix sum? A remainder? A sorted string? A `(row, col)` pair? Choosing the key is most of the problem.
2. **What is the value?** Presence only (a set)? A count? The **first** index? The **last** index? A list of indices?
3. **What do I ask at each step?** "Is the complement there?" "Have I seen this prefix sum before, and how often?" "Where was this character last?"

PROBLEM	KEY	VALUE	QUESTION
Two Sum	value	index	is `target − x` there?
Contains Duplicate II	value	**last** index	was it within k?
Subarray Sum = k	prefix sum	**count**	how many earlier prefixes equal `prefix − k`?
Longest equal 0/1	balance	**first** index	how far back did this balance first appear?
Group Anagrams	letter-count signature	list of words	— (just group)
Longest substring w/o repeat	character	last index	is it inside the window?

First index vs last index vs count is the decision people get wrong most. *Longest* subarray → keep the **first** occurrence (never overwrite). *Nearest / within distance k* → keep the **last** occurrence (always overwrite). *How many* subarrays → keep a **count**.

3.3 Hash table, sorting, or two pointers?

Many hash problems also have a sorting solution. Knowing both, and their trade-offs, is what interviewers look for.

	HASHING	SORT + TWO POINTERS
Time	O(n) average	O(n log n)
Extra space	O(n)	O(1) (in place)
Keeps original indices	yes	no, unless you sort index pairs
Worst case	$O(n^2)$ with adversarial collisions	O(n log n) guaranteed
Handles a stream	yes	no

Two Sum asks for indices — hash it. 3Sum asks for values and must avoid duplicate triplets — sort it. "Do it in O(1) extra space" — sort, or use the array itself (Part 11).

PART 4 — Counting and frequency

4.1 The counting map

```
unordered_map<int, int> cnt;
for (int x : a) cnt[x]++;              // here operator[] is exactly right
```

Everything in this family starts with that line, and then asks something of the counts:

- **Valid Anagram (LC 242)** — the two count tables must be equal. With 26 letters: one array, increment for `s`, decrement for `t`, all zero at the end.
- **Ransom Note (LC 383)** — the magazine is supply, the note is demand; fail as soon as any count goes negative.
- **First Unique Character (LC 387)** — two passes: count, then the first index whose count is 1.
- **Majority element, top-k frequent** — counts first, then pick (Topic 07 for top-k with a heap).
- **Intersection with multiplicity (LC 350)** — count one array, consume counts while scanning the other.

4.2 Counting pairs in one pass

"How many pairs i < j have `a[i] == a[j]`?" — do **not** count first and apply n(n−1)/2 afterwards unless you want to; the one-pass version generalises better:

```
for (int x : a) {
    pairs += cnt[x];       // x pairs with every EARLIER copy of itself
    cnt[x]++;
}
```

Change what you look up and you get a whole family: pairs summing to k (`cnt[k - x]`), pairs with difference k (`cnt[x - k] + cnt[x + k]`), pairs whose sum is divisible by 60 (`cnt[(60 - x % 60) % 60]`, LC 1010).

4.3 Signatures — grouping things that are "the same"

To group items by an equivalence, build a key that **every equivalent item shares and no other item has.** That key is a **signature**.

EQUIVALENCE	SIGNATURE
anagrams	the sorted string — O(L log L); or the 26 counts — O(L)
same set of letters	a 26-bit mask
isomorphic strings / same pattern	first-occurrence indices: `"paper"` → `0,1,0,3,4`
shifted strings (`abc`, `bcd`)	differences between neighbours, mod 26
same row of a grid after moving	the row as a string

```cpp
// Group Anagrams (LC 49) with an O(L) count signature
for (const string& s : strs) {
    string key(26, 0);
    for (char c : s) key[c - 'a']++;      // 26 "characters" holding counts
    groups[key].push_back(s);
}
```

The counts version is O(L) per word instead of O(L log L), and the 26-character string is a perfectly good hash key.

PART 5 — Seen-before and complement lookups

5.1 Two Sum — the most important problem in the topic

The brute force asks, for every pair, whether they sum to the target: O(n²). Turn it around: for each `x`, the partner you need is `target − x` — the **complement**. Has it already appeared?

```
unordered_map<int, int> seen;                    // value -> index
for (int i = 0; i < (int)a.size(); i++) {
    auto it = seen.find(target - a[i]);          // 1. look up the complement
    if (it != seen.end()) return {it->second, i};
    seen[a[i]] = i;                              // 2. then record a[i]
}
```

O(n) time, O(n) space. Learn the shape, because it reappears in half of this guide with different keys.

5.2 Duplicates, and duplicates within a distance

- **Contains Duplicate (LC 217)** — `if (!seen.insert(x).second) return true;` — `insert` returns whether the element was new, so one call checks and records.
- **Contains Duplicate II (LC 219)** — "two equal values at most k apart": store the **last** index of each value and compare. Always overwrite with the newest index; it is the only one that can be close to future elements.
- **Contains Duplicate III** — values within t of each other, indices within k: hash each value into a bucket of width t + 1, so a match must be in the same or a neighbouring bucket. The bucket idea is worth knowing even if you never write it.

5.3 Detecting a cycle with a set

Happy Number (LC 202): repeatedly replace n by the sum of the squares of its digits. Either you reach 1, or the sequence repeats forever. "Repeats" means "*have I seen this number before?*" — a set. (Floyd's fast/slow pointers from Topic 03 solves it in O(1) space; say so.)

5.4 Longest Consecutive Sequence (LC 128) — the O(n) trick

Put every value in a set. A value x starts a run only if `x − 1` is **not** in the set. From each start, walk `x + 1, x + 2, …` while present.

```
for (int x : s) {
    if (s.count(x - 1)) continue;          // not a start: some other x will cover it
    int len = 1;
    while (s.count(x + len)) len++;
    best = max(best, len);
}
```

It looks like O(n²) because of the inner loop. It is **O(n)**: each value is walked over by the inner loop at most once — only from the start of its own run. Iterate over the **set**, not the original array, or a long run of duplicate starts makes it quadratic.

5.5 Meet in the middle — 4Sum II (LC 454)

Four arrays, count tuples summing to 0. Brute force is $O(n^4)$. Hash every `a + b` with its count ($O(n^2)$), then for every `c + d` look up `-(c + d)` ($O(n^2)$). Splitting a search in half and joining the halves through a hash table is called **meet in the middle**, and it is the standard way to turn $O(n^4)$ into $O(n^2)$ or $O(2^n)$ into O(2^(n/2)).

PART 6 — Prefix sums plus a hash map

This is the single most valuable pattern in the topic for placements. Learn it until you can write it without thinking.

6.1 The idea

Let `P[i]` be the sum of the first i elements, with `P[0] = 0`. The sum of the subarray from index j to i − 1 is

```
sum(j .. i-1) = P[i] - P[j]
```

So **"a subarray sums to k" is the same statement as "two prefix sums differ by k."** At position i you need: how many earlier prefixes equal `P[i] − k`? That is a hash map lookup — the Two Sum shape, on prefix sums.

```
a      =    1    2    3                    k = 3
P      = 0    1    3    6

at P = 3: look for 3 - 3 = 0   -> seen once  -> subarray [1,2]
at P = 6: look for 6 - 3 = 3   -> seen once  -> subarray [3]
                                                 total = 2
```

```cpp
unordered_map<int, int> seen{{0, 1}};    // the EMPTY prefix: sum 0, seen once
int prefix = 0, count = 0;
for (int x : a) {
    prefix += x;
    auto it = seen.find(prefix - k);
    if (it != seen.end()) count += it->second;
    seen[prefix]++;
}
```

O(n) — and, unlike the sliding window, it works with **negative numbers**. A window can only shrink or grow in one direction when the values are all positive; prefix sums do not care.

The `{0, 1}` seed is not optional. It stands for the empty prefix before index 0, and without it every subarray that **starts at index 0** is missed.

6.2 The variants, and what changes

PROBLEM	KEY	VALUE STORED	ANSWER
LC 560 count subarrays with sum k	prefix sum	count	`+= count[P − k]`
GfG longest subarray with sum 0 / sum k	prefix sum	**first** index	`i − first[P − k]`
LC 525 longest with equal 0s and 1s	prefix of ±1 ("balance")	first index	`i − first[P]`
LC 974 count subarrays divisible by k	`P mod k`	count	`+= count[P mod k]`
LC 523 a subarray of length ≥ 2, sum multiple of k	`P mod k`	first index	`i − first ≥ 2`
LC 1074 count submatrices with sum target	1-D prefix inside fixed row pairs	count	LC 560 per row pair

Turn the problem into a sum. Equal 0s and 1s becomes "sum zero" once 0 is replaced by −1. "More 1s than 0s", "balanced parentheses substring", "equal vowels and consonants" — all the same move.

Divisible by k means two prefixes with **equal remainders**, because `(P[i] − P[j]) mod k = 0` exactly when `P[i] mod k = P[j] mod k`.

6.3 Negative remainders

In C++, `-7 % 5 == -2`, not `3`. A prefix sum that goes negative then produces a negative remainder that never matches its positive twin, and the count is wrong.

```
prefix = ((prefix + x) % k + k) % k;      // always in 0 .. k-1
```

That one line is the difference between accepted and wrong answer on LC 974.

6.4 Lifting to two dimensions (LC 1074)

To count submatrices summing to a target: fix a pair of rows `top ≤ bottom`, collapse the rows between them into one array of column sums, and run LC 560 on that array. $O(R^2 \cdot C)$. Fixing a pair of boundaries and reducing to the 1-D problem is the standard way to lift any subarray technique to submatrices.

PART 7 — Sliding windows with a hash map

Strings Part 5 covers the sliding window in full. Here is the hashing view: the map holds the **contents of the window**, and you update it as characters enter on the right and leave on the left.

7.1 Longest substring without repeating characters (LC 3)

Store each character's **last index**. When the incoming character was last seen *inside* the window, jump the left edge just past it.

```
vector<int> last(256, -1);
for (int i = 0, left = 0; i < (int)s.size(); i++) {
    if (last[s[i]] >= left) left = last[s[i]] + 1;   // repeat inside the window
    last[s[i]] = i;
    best = max(best, i - left + 1);
}
```

The `>= left` check matters: on `"abba"`, when the final `a` arrives its last index (0) is **outside** the window, and jumping back to 1 would be wrong.

7.2 Fixed-size window: Find All Anagrams (LC 438)

Keep a 26-count table for the window. Each step adds one character and removes one; compare with the pattern's table (26 comparisons, so O(26 · n)). A `matches` counter can make each step O(1).

7.3 Minimum window containing all of t (LC 76)

Track how many required characters are still **missing**. Expand right until `missing == 0`, then shrink left as far as possible while still valid, recording the best window. The counts may go negative — that just means the window holds more copies than needed.

7.4 Exactly k = at most k − at most (k − 1)

"Count subarrays with **exactly** k distinct values" (LC 992) is awkward to window directly, because shrinking does not preserve "exactly". "**At most** k" is easy: for each right edge, every subarray ending there and starting inside the window counts — `right − left + 1` of them. Then

```
exactly(k) = atMost(k) − atMost(k − 1)
```

Use the map's **size** as the distinct count, and erase a key when its count drops to zero — or keep a separate counter as the solution does.

PART 8 — Mappings and bijections

8.1 Isomorphic strings and word pattern

`"egg"` and `"add"` are isomorphic: there is a one-to-one mapping `e→a, g→d`. One map from `s` to `t` is **not enough**:

```
s = "badc",  t = "baba"

s -> t only:   b->b, a->a, d->b, c->a     no conflict seen  ->  "true"   WRONG
t -> s:        b->b, a->a, b->d  CONFLICT                      ->  false
```

`d` and `b` both map to `b`. A **bijection** needs a map in **both directions**, and every pair must agree with both. The same applies to Word Pattern (LC 290): a letter must always mean the same word, **and** a word must always come from the same letter.

8.2 The first-occurrence signature

An alternative that needs no reverse map: replace every character by the index of its **first** occurrence. `"paper"` → `0 1 0 3 4`, `"title"` → `0 1 0 3 4`. Two strings are isomorphic exactly when these sequences are equal. This also gives a key for **grouping** strings by pattern (LC 890 Find and Replace Pattern).

PART 9 — Design problems: O(1) with a hash map plus something else

A hash map gives O(1) lookup **by key** but has no order and no random access by position. Many design problems need one of those as well. The answer is always the same: **pair the hash map with a second structure, and make the map point into it.**

9.1 LRU Cache (LC 146) — hash map + doubly linked list

Requirements: `get` and `put` in O(1), and evict the **least recently used** key when full.

- A **doubly linked list** keeps keys in recency order: most recent at the front, least recent at the back. Moving a node or removing the back is O(1).
- A **hash map from key to the list node** lets you find a key's node in O(1).

```
order (std::list): front = most recent, back = least recent

    ┌─────────┐    ┌─────────┐    ┌─────────┐
    │ 3 : 30  │ ⇄  │ 1 : 10  │ ⇄  │ 2 : 20  │ ◄── evicted next
    └─────────┘    └─────────┘    └─────────┘
        ▲              ▲              ▲
        │              │              │
      key 3          key 1          key 2

where (unordered_map): key -> iterator to its node in the list
```

```cpp
list<pair<int, int>> order;                                  // (key, value)
unordered_map<int, list<pair<int, int>>::iterator> where;

// get: move the node to the front in O(1), without copying it
order.splice(order.begin(), order, where[key]);
```

`list::splice` relinks a node in place, so the iterator stored in the map **stays valid**. That is why `std::list` (not `vector`) is the right container: its iterators survive insertions and removals elsewhere in the list. On eviction, erase the **map entry first** (you need the key from the back node), then pop the list.

LRU is the single most-asked design question in on-campus interviews. Be able to write it from a blank page in 15 minutes, and also to explain it with a hand-written node struct in case the interviewer forbids `std::list`.

9.2 Insert, Delete, GetRandom in O(1) (LC 380)

`getRandom` needs random access by position — a **vector**. `remove` needs to find a value's position — a **map from value to index**. Deleting from the middle of a vector is O(n)... unless you **move the last element into the hole** and pop the back:

```cpp
int i = pos[val], last = vals.back();
vals[i] = last;  pos[last] = i;          // the last element takes val's place
vals.pop_back(); pos.erase(val);         // erase val LAST — it may BE the last element
```

Order is not preserved, and the problem does not ask for it. The duplicates-allowed version (LC 381) maps each value to a **set of indices**.

9.3 LFU Cache (LC 460)

Evict the **least frequently** used key, breaking ties by least recently used. Three structures: `key → (value, freq, iterator)`, `freq → list of keys in recency order`, and a running `minFreq`. A new key always resets `minFreq` to 1; touching a key moves it from list `f` to list `f + 1`, and bumps `minFreq` if list `f` became empty and was the minimum.

9.4 Design HashMap (LC 706)

The problem *is* Part 1: implement put / get / remove without the library. An array of buckets with chaining, a prime bucket count, and overwrite-on-put. Expect the follow-ups: *what is your load factor?*, *how would you resize?*, *how would you handle deletion with open addressing?*

PART 10 — Hashing strings: rolling hashes

10.1 Why a rolling hash

Comparing two strings of length L costs O(L). If you need to compare **every window** of length L in a text against something — a pattern, or each other — that is O(n · L). A **rolling hash** computes the hash of the next window from the previous one in **O(1)**.

10.2 The polynomial rolling hash

With base B and modulus M:

```
hash("c0 c1 … c(L-1)") = c0·B^(L-1) + c1·B^(L-2) + … + c(L-1)     (mod M)

slide right by one:  drop c0, add cL
h_new = (h_old − c0·B^(L-1)) · B + cL                               (mod M)
```

Precompute `B^(L−1)` once. Keep every intermediate value in `long long` and add M before taking `%` after a subtraction so the result stays non-negative.

10.3 Rabin-Karp: find every occurrence of a pattern

Hash the pattern, roll a hash over every window of the text, and **when the hashes match, compare the actual characters** before reporting. Average O(n + m). Two strings with the same hash are only *probably* equal.

10.4 Collisions, and how to make them unlikely

With M ≈ 10^9 and N windows, the *birthday paradox* says some pair of different windows collides once N reaches about √M ≈ 30 000. With 10^5 windows, a collision is likely. Three defences:

- **Verify on a hash match** (Rabin-Karp, LC 1044 in the solved-questions book) — always correct, slower only when there are many true matches.
- **Double hashing** — two independent moduli (e.g. $10^9 + 7$ and $10^9 + 9$); a false match needs both to collide, probability ≈ 1 / 10^{18}.
- **Modulus 2^{64}** via `unsigned long long` overflow — fast, but a known string family (Thue–Morse) breaks it for *any* base. Use it only with verification.

10.5 When the hash can be perfect

Repeated DNA Sequences (LC 187): four letters need two bits each, so a 10-letter window fits exactly in 20 bits. Shift in two bits, mask off the oldest two, and the integer *is* the window — no collisions possible. Whenever the alphabet is tiny and the window is short, encode instead of hashing.

10.6 Binary search on the answer + rolling hash

Longest Duplicate Substring (LC 1044): if some substring of length L appears twice, so does one of length L − 1 (drop its last character). That monotonicity allows **binary search on L**; each check is

"does any window of length L repeat?", answered with a rolling hash and a set in O(n). Total O(n log n). This pairing appears in several hard string problems.

PART 11 — The array as its own hash table

When the values are known to lie in **1 … n** and the problem demands **O(1) extra space**, the array's own indices can serve as the buckets: value v "belongs" at index v − 1.

11.1 Cyclic placement — First Missing Positive (LC 41)

Swap every value in 1 … n into its home slot, ignoring values out of range and duplicates. Afterwards, the first index i whose slot does not hold i + 1 gives the answer.

```
for (int i = 0; i < n; i++)
    while (a[i] >= 1 && a[i] <= n && a[a[i] - 1] != a[i])
        swap(a[i], a[a[i] - 1]);
for (int i = 0; i < n; i++) if (a[i] != i + 1) return i + 1;
return n + 1;
```

O(n): every swap puts one value into its final home, so there are at most n swaps in total. The condition `a[a[i] − 1] != a[i]` (not `a[i] != i + 1`) is what stops an infinite loop on duplicates.

11.2 Sign marking

To find duplicates or missing numbers in 1 … n without extra space: for each value v, flip `a[|v| − 1]` to negative. A slot that is already negative means v was seen before; a slot still positive at the end means its number never appeared. (LC 442 Find All Duplicates, LC 448 Find All Disappeared Numbers.) Restore the signs afterwards if the input must be preserved.

PART 12 — The complete pattern taxonomy

Every problem in Part 16 is labelled with one of these twelve. When you can name the pattern, you already have most of the code.

#	PATTERN	RECOGNISE IT BY	CORE TOOL
1	Existence / duplicate check	"seen before", "duplicate", "unique"	`unordered_set`, `insert(x).second`
2	Frequency counting	"how many times", anagram, supply vs demand	count map or `int cnt[26]`
3	Complement lookup	"two values that add up / differ by k"	value → index; look up, then insert
4	Last / first position	"within distance k", "longest"	value → last index, or → first index
5	Signature grouping	"group", "same pattern", "rearranged"	a canonical key shared by equivalent items
6	Bijection	isomorphic, word pattern	two maps, one each direction
7	Prefix sum + hash map	"subarray with sum k", equal 0/1, divisible by k	`count[P − k]`, seeded with `{0: 1}`
8	Sliding window + map	"longest substring with …", "all anagrams of p"	window contents in a map
9	Set-based run walking	"longest consecutive", "start of a sequence"	start only where `x − 1` is absent
10	Design with map + list / vector	LRU, LFU, O(1) getRandom	map points into the second structure
11	Rolling hash	repeated substring, pattern search	polynomial hash, O(1) slide, verify
12	Array as hash table	values in 1 … n, O(1) extra space	cyclic placement or sign marking

How to tell 3 and 7 apart

Both look up "the thing that would complete the answer."

- **Pattern 3** looks up a complementary **element**: `target − x`.
- **Pattern 7** looks up a complementary **prefix sum**: `P − k`. The answer is a *subarray*, not a pair of elements.

If the problem says "subarray" or "contiguous", think prefix sums. If it says "pair" or "two numbers", think complement.

How to tell 7 and 8 apart

Both find subarrays.

- **Sliding window** needs the window condition to be **monotonic** — growing the window can only make it "more" violated, shrinking only "less". That holds for "sum ≤ k with positive numbers" or "at most k distinct". O(1) extra space with arrays.
- **Prefix sum + map** works when it is not monotonic — **negative numbers**, "exactly k", "divisible by k".

If the array can contain negatives and the question is about sums, the sliding window is wrong. Use prefix sums.

PART 13 — How to approach ANY hashing problem

Six steps. They prevent the three expensive mistakes: the wrong key, storing first-index when you needed last-index (or a count), and an `operator[]` that quietly inserts.

Step 1 — Write the brute force and find the inner search

Write the O(n^2) version in your head. Find the inner loop that asks "*is there an earlier element / prefix / window such that …?*" That inner loop is what the hash table replaces.

Step 2 — Choose the key

Value? Prefix sum? Remainder? Signature? Pair encoded as a `long long`? If the keys are small integers or letters, use an **array** instead of a map.

Step 3 — Choose the value: presence, count, first index or last index

- "does one exist?" → presence (set)
- "how many?" → count
- "longest" → **first** index; never overwrite
- "nearest" / "within k" → **last** index; always overwrite

Step 4 — Decide the seed and the order

Does the table need an initial entry? Prefix-sum problems need `{0: 1}` (counting) or `{0: −1}` (longest). Then decide: **look up before insert** (pairs i < j, never pairing an element with itself) — almost always.

Step 5 — Guard the lookups

Read with `find` or `count`, write with `[]`. Never `if (m[x])` on a key that might be absent. Normalise negative remainders. Use `long long` for prefix sums that can exceed 2^{31}.

Step 6 — Dry-run the small cases

1. **Empty input** and a **single element**.
2. **All elements equal** — catches first-vs-last index and the "pair with itself" bug.
3. **A subarray starting at index 0** — catches a missing `{0: …}` seed.
4. **Negative numbers** — catches a sliding window used where prefix sums are needed, and negative `%`.
5. **No answer exists**.

PART 14 — The bug catalogue

1. `operator[]` **inserting on a read.** `if (m[x] > 0)` inserts `x` with 0 when absent. Symptom: `m.size()` is wrong, distinct counts are inflated, iteration sees extra keys, or the map grows until MLE. Read with `find` / `count`.

2. Inserting before looking up. In Two Sum, `seen[a[i]] = i` before the lookup pairs an element with itself: `[3, 2, 4]`, target 6 returns `[0, 0]`. Symptom: a self-pair. Look up first.

3. Missing the `{0: 1}` **/** `{0: -1}` **seed in prefix-sum problems.** Symptom: every subarray that starts at index 0 is missed — `[1, 1, 1]` with k = 2 gives 1 instead of 2.

4. Overwriting the first index when you needed the longest. In LC 525 / longest zero-sum, `first[P] = i` on every step makes every length 0. Only store when absent.

5. Negative remainders. `-7 % 5 == -2` in C++. Symptom: wrong counts in "divisible by k" problems as soon as a prefix goes negative. Use `((p % k) + k) % k`.

6. A sliding window on an array with negative numbers. Shrinking no longer helps monotonically. Symptom: wrong answers on inputs with negatives. Use prefix sums + hash map.

7. Not erasing a key whose count drops to zero. Symptom: `map.size()` over-counts distinct values in a window.

8. Only one map for a bijection. Symptom: `"badc"` / `"baba"` reported isomorphic.

9. Hashing a type that has no `std::hash`**.** `unordered_map<pair<int,int>, int>` does not compile. Encode as `long long`, use `map`, or write a hash functor.

10. Erasing while iterating with a stale iterator. `m.erase(it); ++it;` is undefined behaviour. Use `it = m.erase(it);`.

11. Integer overflow in prefix sums or encoded keys. Prefix sums of 10^5 values up to 10^9 need `long long`; so does `r * 100000 + c` when r and c are large.

12. Trusting a hash match without verifying. Different strings can share a rolling hash. Symptom: rare wrong answers that pass most tests. Verify on a match, or double-hash.

13. Iterating the original array in Longest Consecutive Sequence. With many duplicates of a run start, the inner walk repeats. Iterate the **set**. Symptom: TLE on large inputs with duplicates.

14. In LRU, erasing the list node before reading its key. `order.pop_back();` `where.erase(order.back().first);` erases the wrong key. Read the key, erase from the map, then pop.

15. Assuming an iteration order. `unordered_map` iterates in no particular order, and the order changes between compilers and after rehashes. Symptom: output that matches locally and fails on the judge when the order matters. Sort, or use `map`.

16. Relying on `unordered_map` **when the judge hacks.** On Codeforces, crafted keys make every operation O(n). Use a randomised hash (Part 2.6) or `map`.

PART 15 — Complexity cheat sheet

OPERATION / ALGORITHM	TIME	SPACE	NOTE
`unordered_set` / `unordered_map` insert, find, erase	**O(1) avg**, O(n) worst	O(n)	amortised over rehashes
`set` / `map` insert, find, erase	O(log n)	O(n)	ordered, `lower_bound`
array as a map (`int cnt[26]`)	O(1) worst	O(alphabet)	fastest when keys are small
rehash	O(n)	O(n)	amortised O(1) per insert
Two Sum, contains duplicate, frequency count	O(n)	O(n)	
Group anagrams	O(n · L) with count keys	O(n · L)	O(n · L log L) with sorted keys
Longest consecutive sequence	O(n)	O(n)	start only at run starts
Subarray sum = k (prefix + map)	O(n)	O(n)	works with negatives
Submatrix sum = target	$O(R^2 \cdot C)$	O(C)	fix two rows, LC 560 on columns
Sliding window with a map	O(n)	O(alphabet)	
4Sum II (meet in the middle)	$O(n^2)$	$O(n^2)$	from $O(n^4)$
LRU / LFU cache operations	O(1)	O(capacity)	map + list
Rabin-Karp	O(n + m) average	O(1)	O(n · m) worst with many matches
Longest duplicate substring	O(n log n)	O(n)	binary search + rolling hash
First missing positive	O(n)	**O(1)**	the array is the table

The sentences to say in interviews:

> *"I trade O(n) extra space for time: a hash map of what I have already seen turns the inner search into an O(1) lookup, so the whole thing is O(n) on average."*
>
> *"A subarray sum is a difference of two prefix sums, so I count how many earlier prefixes equal the current prefix minus k. That handles negative numbers, which a sliding window cannot."*
>
> *"Hash operations are O(1) on average and O(n) in the worst case; if I need a guaranteed bound or sorted order, I would use `std::map` at O(log n)."*

PART 16 — The problem list

Work these in order. Every one is solved in the companion **Hashing — Solved Questions** PDF, in exactly this order — but attempt each problem yourself first and only read the solution after you have genuinely struggled with it.

Tier 1 — Learn the mechanics (do all thirteen first)

#	PROBLEM	PATTERN
1	Two Sum	3 — complement lookup ⭐
217	Contains Duplicate	1 — existence
242	Valid Anagram	2 — counting ⭐
383	Ransom Note	2 — supply vs demand
387	First Unique Character in a String	2 — two-pass count
205	Isomorphic Strings	6 — bijection ⭐
290	Word Pattern	6 — bijection
349	Intersection of Two Arrays	1 — existence
350	Intersection of Two Arrays II	2 — counts consumed
202	Happy Number	1 — cycle detection
219	Contains Duplicate II	4 — last index
1512	Number of Good Pairs	2 — pairs in one pass
706	Design HashMap	— build the table by hand

Thirteen problems. When you can write Two Sum, Valid Anagram and Isomorphic Strings from a blank page — and explain why Isomorphic needs two maps — you have the mechanics.

Tier 2 — The placement core (these are what actually get asked)

#	PROBLEM	PATTERN
49	Group Anagrams	5 — signature ⭐
128	Longest Consecutive Sequence	9 — run starts ⭐
560	Subarray Sum Equals K	7 — prefix sum + map ⭐
525	Contiguous Array	7 — ±1 balance, first index
974	Subarray Sums Divisible by K	7 — remainders
454	4Sum II	3 — meet in the middle
36	Valid Sudoku	1 — three "seen" tables
380	Insert Delete GetRandom O(1)	10 — map + vector ⭐
146	LRU Cache	10 — map + list ⭐
438	Find All Anagrams in a String	8 — fixed window
3	Longest Substring Without Repeating Characters	8 — last index in the window ⭐
523	Continuous Subarray Sum	7 — remainder, first index
1010	Pairs of Songs With Total Durations Divisible by 60	3 — complement remainder
554	Brick Wall	2 — count edge positions
187	Repeated DNA Sequences	11 — perfect rolling encoding

⭐ **Two Sum, Group Anagrams, Subarray Sum Equals K, Longest Consecutive Sequence, Longest Substring Without Repeating Characters and LRU Cache are the most-asked hashing questions in on-campus drives** — Amazon, Microsoft, Adobe, Flipkart, Paytm, Goldman Sachs, TCS Digital, Infosys. Be able to write all six cold.

Tier 3 — Harder, but they do appear

#	PROBLEM	PATTERN
41	First Missing Positive	12 — cyclic placement
76	Minimum Window Substring	8 — need / missing counter
30	Substring with Concatenation of All Words	8 — word-sized windows
149	Max Points on a Line	5 — reduced slope as key
460	LFU Cache	10 — map + list per frequency
1074	Number of Submatrices That Sum to Target	7 — LC 560 in 2-D
992	Subarrays with K Different Integers	8 — exactly = atMost − atMost
336	Palindrome Pairs	5 — reversed words as keys
1044	Longest Duplicate Substring	11 — binary search + rolling hash
381	Insert Delete GetRandom O(1) — Duplicates allowed	10 — map to index sets

The classic Indian-placement questions (GfG / written-round favourites)

These appear in written rounds and technical interviews, usually in GfG's wording:

- **Explain hashing, collisions, chaining and open addressing** — and why deletion needs tombstones
- **What is the load factor? When and how does a table rehash?**
- Implement a hash table with chaining; implement linear probing
- **Count the frequency of every element**; find the most / least frequent
- **First repeating element**; first non-repeating element
- **First non-repeating character in a stream**
- Check whether two arrays are equal (same elements, any order)
- **Pair with given sum** (Two Sum) — count pairs, print pairs
- **Count pairs with given sum**; pairs with difference k
- **Largest subarray with 0 sum**; longest subarray with sum k
- **Count subarrays with sum k**; count subarrays with equal 0s and 1s
- **Count distinct elements in every window of size k**
- **Longest consecutive subsequence**
- Union and intersection of two arrays
- Check whether an array is a subset of another
- **Group anagrams together**; check if two strings are anagrams
- Itinerary from a list of tickets (find the start, then follow the map)
- Find the four elements that sum to a value (4Sum with pair sums in a map)
- Check if a subarray with sum 0 exists

- Zero-sum subarrays — print all of them
- **LRU cache**

> **"Largest subarray with 0 sum"** and **"count subarrays with given sum"** are close to guaranteed in some form. Both are Part 6 — prefix sums with a hash map — and the most common wrong answer is forgetting the empty prefix: `{0: -1}` for the longest, `{0: 1}` for the count.

PART 17 — A three-week plan

Assumes about 1.5 hours a day.

Week 1 — The structure and the basics. Days 1–2: Parts 0–2, then practice TODOs 1–3 **without the STL**: the chained map with rehashing, open addressing with tombstones, and Rabin-Karp. Day 3: Part 3, and write the first-index / last-index / count table from memory. Days 4–7: Tier 1 in order — 1, 217, 242, 383, 387, 205, 290, 349, 350, 202, 219, 1512, 706 — and practice TODOs 4–6.

Week 2 — Prefix sums and windows. This is the week that matters. Days 1–3: Part 6 — 560, 525, 974, 523, then the GfG largest-zero-sum subarray; practice TODOs 7–8. Write the 6-line LC 560 loop from memory on three separate days. Days 4–5: Part 7 — 3, 438, then 76. Days 6–7: 49, 128, 454, 1010, 554, 36; practice TODOs 9–10.

Week 3 — Design and hard. 380 and 146 back to back, then 460 and 381. LRU is the one to drill: write it on paper twice. Then 187 and Part 10, then 1044. Then 41, 30, 149, 1074, 992, 336 as far as you get. Finish by re-solving **1, 49, 560, 128, 3 and 146** on a timer — 15 minutes each, on paper, no compiler.

Ongoing: every time a problem defeats you, log which of the 12 patterns it was and which of the bugs you hit. After thirty problems that log tells you exactly what to drill.

Where this topic leads: the sliding window is shared with Strings (Topic 02). Graphs (Topic 12) store visited sets and adjacency in hash maps. Memoisation in Dynamic Programming (Topic 11) is a hash map from state to answer. Tries (Topic 14) are the alternative to hashing when keys are strings and prefixes matter. LRU and LFU return in Design & LLD (Topic 15). Nothing here is wasted.

Quick reference card

```
// ===== containers =====
unordered_map<K, V> m;  unordered_set<K> s;     // O(1) avg, any order
map<K, V> om;           set<K> os;              // O(log n), sorted, lower_bound
int cnt[26] = {0};      cnt[c - 'a']++;         // when keys are small: an array

// ===== read safely: [] INSERTS =====
auto it = m.find(k);  if (it != m.end()) use(it->second);
if (m.count(k)) ...
if (!s.insert(x).second) /* x was already there */;
for (auto it = m.begin(); it != m.end(); ) it = cond ? m.erase(it) : next(it);

// ===== complement (Two Sum) — look up, THEN insert =====
for (int i = 0; i < n; i++) {
    auto it = seen.find(target - a[i]);
    if (it != seen.end()) return {it->second, i};
    seen[a[i]] = i;
}

// ===== prefix sum + map =====
unordered_map<long long, int> cnt{{0, 1}};      // COUNT subarrays: seed {0: 1}
for (int x : a) { p += x; ans += cnt.count(p - k) ? cnt[p - k] : 0; cnt[p]++; }

unordered_map<long long, int> first{{0, -1}};   // LONGEST subarray: seed {0: -1}
for (int i = 0; i < n; i++) { p += a[i];
    if (first.count(p - k)) best = max(best, i - first[p - k]);
    if (!first.count(p)) first[p] = i; }        // never overwrite

r = ((p % k) + k) % k;                          // remainder, never negative

// ===== signature =====
string key(26, 0); for (char c : w) key[c - 'a']++; groups[key].push_back(w);

// ===== longest consecutive =====
for (int x : s) if (!s.count(x - 1)) { int len = 1; while (s.count(x + len)) len++; best = max(best,
len); }

// ===== LRU =====
list<pair<int,int>> order;  unordered_map<int, list<pair<int,int>>::iterator> where;
order.splice(order.begin(), order, where[key]);          // touch, O(1)
where.erase(order.back().first); order.pop_back();       // evict: map first

// ===== O(1) remove from a vector: swap with last =====
int i = pos[v], last = vals.back(); vals[i] = last; pos[last] = i;
vals.pop_back(); pos.erase(v);

// ===== rolling hash =====
h = (h * B + c) % M;                                     // append
h = ((h - out * pw % M + M) * B + in) % M;               // slide, pw = B^(L-1)

// ===== pair keys =====
long long key = (long long)r * 1000003 + c;              // or map<pair<int,int>, V>
```

The five rules, one last time:

1. **Hashing trades memory for time.** If the brute force searches earlier elements, hash them.
2. **Choose the key, then the value** — presence, count, **first** index (longest) or **last** index (nearest).
3. **Look up, then insert.** That keeps pairs i < j and never pairs an element with itself.
4. `[]` **writes,** `find` **reads.** `m[x]` on a missing key inserts it.

5. **Subarray + sum → prefix sums + map, seeded with the empty prefix.** It handles negatives; a sliding window does not.